



Saint Columban College
College of Computing Studies
Pagadian City



Even RSA can be broken???

EVEN RSA CAN BE BROKEN???



Easy Cryptography picoCTF 2025 browser_webshell_solvable

AUTHOR: MICHAEL CROTTY

Description

This service provides you an encrypted flag. Can you decrypt it with just N & e?

Additional details will be available after launching your challenge instance.

This challenge launches an instance on demand.

Its current status is: **NOT_RUNNING**

Launch Instance

Hints ?

1 2 3

Notice anything interesting about N?

20,918 users solved



95% Liked



picoCTF{FLAG}

Submit Flag

Author: The Analyst: Hyposelenia

Challenge: EVEN RSA CAN BE BROKEN???

Category: Cryptography

Date: 02/25/26



I. Objective

The objective of this challenge is to **understand how RSA encryption works** and to demonstrate that **RSA can be broken if it is implemented incorrectly**, allowing us to recover the encrypted message and obtain the flag.

II. Background

RSA is one of the **most famous and widely used encryption algorithms** in the world. It protects:

- Websites (HTTPS)
- Messages
- Logins
- Secure communications

RSA is considered **very secure** — but **only when used properly**. In Capture The Flag (CTF) challenges, RSA is often intentionally:

- Too small
- Misconfigured
- Missing secret components

This challenge teaches an important cybersecurity lesson:

Strong cryptography can become weak if implemented carelessly.

III. Tool Used

- **Kali Linux (VirtualBox)** – Used as the Linux environment
- **nc (netcat)** – Used to connect to the remote RSA challenge server
- **Online RSA Decoder** – Used to compute the plaintext from public values



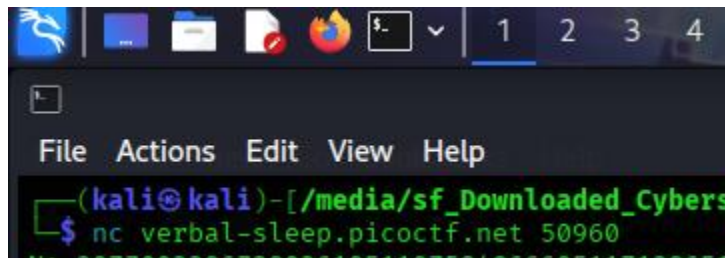
IV. Methodology

1. Launched the challenge instance.

Its current status is: **NOT_RUNNING**

Launch Instance

2. Opened Kali Linux using Oracle VirtualBox.
3. Connected to the challenge server using the provided nc command after launching in the terminal.



4. Observed the values given by the server:

- a. n (modulus)
- b. e (public exponent)
- c. ciphertext

```
(kali@kali)-[/media/sf_Downloaded_Cybersecfiles]
$ nc verbal-sleep.picocft.net 50960
N: 20770033867280361051107594866605117138651818363060303948438770684059433051584735809664277840746999565171124815796297078370729909325942529917964869500515186
e: 65537
ciphertext: 3657401157083089590606302326169046028998106368773454121269073805197020860842375500219461425316004196472679427872764868187051916505911048329897201555529781
(kali@kali)-[/media/sf_Downloaded_Cybersecfiles]
$
```

5. Copied each value carefully from the terminal.

(Ctrl + Shift + C) if at bash



6. Opened an online RSA decoder tool.
7. Pasted:
 - a. n into the modulus field
 - b. e into the exponent field
 - c. ciphertext into the ciphertext field

RSA DECODER

Indicate known numbers, leave remaining cells empty.

★ VALUE OF THE CIPHER MESSAGE (INTEGER) C=

3657401157083089590606302326169046028998106368773...

★ PUBLIC KEY E (USUALLY E=65537) E=

65537

★ PUBLIC KEY VALUE (INTEGER) N=

2077003386728036105110759486660511713865181836306...

★ PRIVATE KEY VALUE (INTEGER) D=

★ FACTOR 1 (PRIME NUMBER) P=

★ FACTOR 2 (PRIME NUMBER) Q=

★ INTERMEDIATE VALUE PHI (INTEGER) Φ =

★ DISPLAY ☒ PLAINTEXT AS CHARACTER STRING

8. Decoded the ciphertext.

★ DISPLAY ☒ PLAINTEXT AS CHARACTER STRING

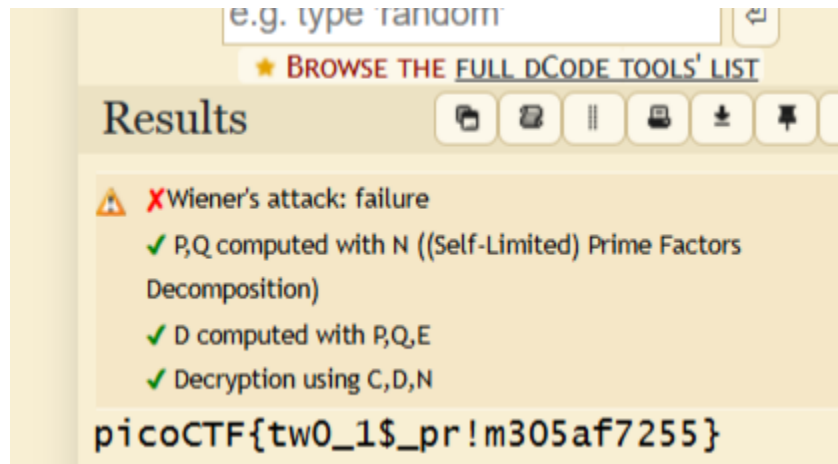
☐ COMPUTED VALUES (C,D,E,N,P,Q,...)

☐ PLAINTEXT AS INTEGER NUMBER

☐ PLAINTEXT AS HEXADECIMAL FORMAT

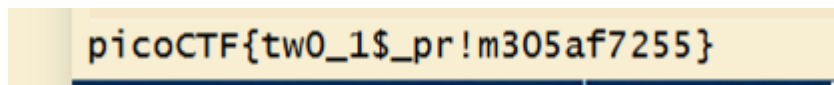


9. Retrieved the decrypted message containing the flag.



V. Result

picoCTF{tw0_1\$_pr!m305af7255}



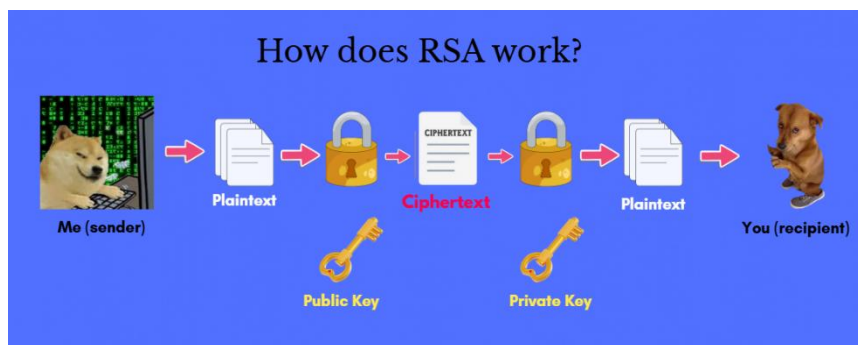
VI. Explanation

What is RSA?

Imagine a **transparent mailbox**:

- Anyone can **drop/see a letter inside**
- Only the owner can **open it**

That mailbox is **RSA encryption**.





What does “Asymmetric” mean?

Asymmetric encryption means:

- One key to **lock**
- A different key to **unlock**

RSA uses:

- **Public Key** → used to encrypt (shared with everyone)
- **Private Key** → used to decrypt (kept secret)

What is n?

n is a **big number** made by multiplying **two secret prime numbers**:

$$n = p \times q$$

- p and q are **prime numbers**
- They must stay secret

If someone can find p and q, RSA is broken.

What are primes?

Prime numbers:

- Can only be divided by **1 and themselves**
- Examples: 2, 3, 5, 7, 11

RSA security depends on primes being **very large**.

What is the totient (ϕ)?

The **totient** is calculated as:

$$\phi(n) = (p - 1)(q - 1)$$

It helps generate the **private key**.

If someone knows $\phi(n)$, they can decrypt messages.



What is e ?

e is the *public exponent*.

It is:

- Part of the public key
- Used during encryption

Common values: 3, 17, 65537

What is c (ciphertext)?

c is the *encrypted message*.

Encryption formula:

$$c = m^e \bmod n$$

Where:

- m = original message
- e = public exponent
- n = modulus

Why was RSA breakable here?

In **real RSA**, n is:

- 2048 bits or larger
- Impossible to factor with normal computers

In this challenge:

- n was **too small**
- The primes were easy to factor
- Online tools could compute the private key



This makes RSA **weak by design** for learning purposes.

Important clarification

RSA itself is not broken.

What's broken is:

- Small key size
- Poor implementation
- Weak parameters

This challenge proves:

Cryptography is only as strong as its implementation.

Wait... How about we try manually! Let's try! Manually solving RSA, for understading!

Step 1: Choose two prime numbers (p and q)

We pick **small primes** on purpose (for learning).

- $p = 11$
- $q = 13$

Both are **prime** (only divisible by 1 and themselves)

Step 2: Compute n

$$n = p \times q$$

$$n = 11 \times 13 = 143$$

n is part of the **public key**



Step 3: Compute Euler's Totient $\phi(n)$:

Formula:

$$\phi(n) = (p - 1)(q - 1)$$

So:

$$\phi(n) = (11 - 1)(13 - 1)$$

$$\phi(n) = 10 \times 12 = 120$$

Step 4: Choose public exponent e

Rules:

- $1 < e < \phi(n)$
- $\gcd(e, \phi(n)) = 1$ (they share no factors)

Let's choose:

$$e = 7$$

Check:

- $\gcd(7, 120) = 1$

So **public key** = ($n = 143$, $e = 7$)

Step 5: Encode a message

Convert letters to numbers

We use a simple alphabet rule, we'll call it **A1Z26** system:

Letter	Number
A	1
R	18
S	19



So:

S A R A

19 1 18 1

We will encrypt **each number separately**.

Step 6: Encrypt (Encoding → Ciphertext)

RSA encryption formula:

$$C = M^e \bmod n$$

Encrypt S (19)

$$C = 19^7 \bmod 143$$

$$C = 46$$

Encrypt A (1)

$$C = 1^7 \bmod 143 = 1$$

Encrypt R (18)

$$C = 18^7 \bmod 143 = 57$$

Encrypt A (1)

$$C = 1$$

Ciphertext Result

The encrypted message becomes:

[46, 1, 57, 1]

Now Let's Break It (Manual Decryption)

Step 7: Compute the private key d



Well, you could've compute the private key after computing the public key.

We solve:

$$d \times e \equiv 1 \pmod{\varphi(n)}$$

So:

$$d \times 7 \equiv 1 \pmod{120}$$

The value that works:

$$d = 103$$

(Yes, this can be solved manually or via Extended Euclidean Algorithm.)

Step 8: Decrypt the ciphertext

RSA decryption formula:

$$M = C^d \pmod{n}$$

Decrypt 46

$$46^{103} \pmod{143} = 19 \rightarrow S$$

Decrypt 1

$$1 \rightarrow A$$

Decrypt 57

$$57^{103} \pmod{143} = 18 \rightarrow R$$

Decrypt 1

$$1 \rightarrow A$$

Plaintext: Sara

Computers make and secure it way better than us lolol.



Saint Columban College
College of Computing Studies
Pagadian City



VII. Conclusion

This challenge demonstrates that **even strong encryption algorithms like RSA can be compromised if used incorrectly**. By exposing weak parameters such as small prime values, the encryption becomes solvable and insecure. Understanding how RSA works internally is essential for both breaking insecure systems in CTFs and building secure systems in real life.

— **The Analyst: Hyposelenia**